

1. ANÁLISIS DE RENDIMIENTO

con SQL Profiler y Extended Events

Herramientas de SQL Server que permiten monitorear, capturar y analizar la actividad del servidor para detectar problemas de rendimiento y optimizar el funcionamiento de la base de datos.

¿QUÉ ES EL ANÁLISIS DE RENDIMIENTO?

Es el proceso de evaluar el funcionamiento de una base de datos para identificar problemas que provocan lentitud, alto consumo de recursos o afectación del servidor. El objetivo es encontrar "cuellos de botella" y optimizar el sistema.

¿CUÁNDO SE UTILIZA?

- Cuando una consulta demora demasiado.
- Cuando el servidor está lento.
- Para detectar errores.
- Para analizar bloqueos (locks).
- Para encontrar consultas que consumen mucha CPU o recursos.

SQL PROFILER

Herramienta gráfica que permite capturar en tiempo real toda la actividad que ocurre dentro del servidor.

Permite observar:

- Consultas ejecutadas.
- Tiempo de ejecución.
- Usuario que ejecutó la consulta.
- Base de datos utilizada.
- Errores.
- Bloqueos (locks).
- Procedimientos almacenados.

Ventajas

- Fácil de usar.
- Permite monitoreo en tiempo real.
- Muy útil para aprendizaje y pruebas.

Desventajas

- Consume recursos del servidor.
- No es recomendable mantenerlo ejecutándose en producción durante mucho tiempo.
- Microsoft recomienda reemplazarlo por Extended Events.

EXTENDED EVENTS

Es la herramienta moderna de monitoreo de SQL Server. Permite recopilar información sobre eventos específicos utilizando menos recursos que SQL Profiler.

Puede registrar:

- Consultas lentas.
- Deadlocks (bloqueos mutuos).
- Errores.
- Uso de CPU.
- Lecturas y escrituras.
- Esperas (Waits).

Ventajas

- Consume menos recursos.
- Más flexible y personalizable.
- Recomendada por Microsoft para entornos de producción.
- Permite capturar solo los eventos necesarios.

DIFERENCIAS PRINCIPALES

	SQL PROFILER	EXTENDED EVENTS
Generación	Más antiguo	Más moderno
Consumo de recursos	Consume más recursos	Consume menos recursos
Interfaz	Sencilla y directa	Más flexible y configurable
Uso en producción	No recomendado	Recomendado por Microsoft
Facilidad para principiantes	Fácil de aprender	Requiere más conocimiento pero es más potente

¿SABÍAS QUÉ?

Así como el "mate" es compartido para mejorar la charla, estas herramientas comparten información clave para mejorar el rendimiento de tu base de datos.

Monitorear es conocer, analizar es entender, optimizar es mejorar.

2. ESTADÍSTICAS E ÍNDICES

(CREACIÓN, FRAGMENTACIÓN Y MANTENIMIENTO)

Los índices y las estadísticas son fundamentales para que SQL Server encuentre y procese los datos de manera rápida y eficiente. Su mantenimiento adecuado garantiza consultas más rápidas y un mejor rendimiento.

¿QUÉ SON LOS ÍNDICES?

Un índice funciona como el índice de un libro.
Sin índice: el motor debe revisar todas las filas.
Con índice: el motor encuentra la información rápidamente.



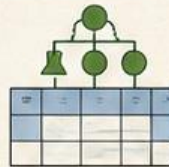
Ejemplo:

Buscar un cliente por NIF (número fiscal).
Con índice tarda milisegundos.
Sin índice revisa toda la tabla.

TIPOS PRINCIPALES DE ÍNDICES

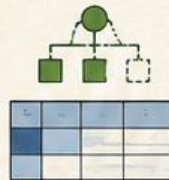
ÍNDICE CLUSTERED

- ✓ Ordena físicamente la tabla según el índice.
- ✓ Solo puede existir uno por tabla.



ÍNDICE NONCLUSTERED

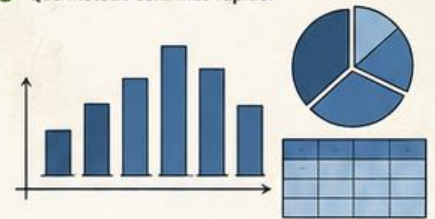
- ✓ No cambia el orden físico de la tabla.
- ✓ Se pueden crear varios por tabla.



¿QUÉ SON LAS ESTADÍSTICAS?

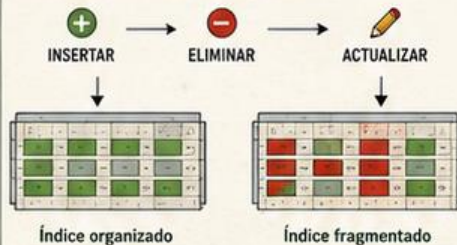
Las estadísticas almacenan información sobre cómo están distribuidos los datos en las columnas e índices. El optimizador utiliza esas estadísticas para decidir:

- ✓ Qué índice usar.
- ✓ Qué plan de ejecución generar.
- ✓ Qué método será más rápido.



FRAGMENTACIÓN DE ÍNDICES

Con el tiempo, las operaciones de INSERT, DELETE y UPDATE hacen que los índices se desorganicen.



Consecuencias:

- Consultas más lentas.
- Más lecturas del disco.
- Mayor uso de memoria y CPU.

MANTENIMIENTO DE ÍNDICES Y ESTADÍSTICAS

REORGANIZAR ÍNDICE

- Se usa cuando la fragmentación es baja (5% - 30%).
- No elimina el índice.
- Solo lo reorganiza y mejora el rendimiento sin bloquear la tabla por mucho tiempo.

RECONSTRUIR ÍNDICE

- Se usa cuando la fragmentación es alta (mayor a 30%).
- Elimina y crea nuevamente el índice.
- Mejora considerablemente el rendimiento.

ACTUALIZAR ESTADÍSTICAS

- Mantiene informado al optimizador.
- Se recomienda hacerlo periódicamente o cuando hay muchos cambios en los datos.

BENEFICIOS

- Consultas más rápidas.
- Menor consumo de CPU.
- Mejor rendimiento general del sistema.
- Menos acceso al disco.
- Mayor capacidad de usuarios concurrentes.

BUENAS PRÁCTICAS

- ✓ Crear índices solo sobre columnas que se usan en WHERE, JOIN, ORDER BY o GROUP BY.
- ✓ No crear índices innecesarios: pueden afectar el rendimiento en INSERT, UPDATE y DELETE.
- ✓ Monitorear la fragmentación periódicamente.
- ✓ Actualizar estadísticas con regularidad.
- ✓ Realizar mantenimiento automático con trabajos programados (SQL Agent).

COMANDOS ÚTILES (T-SQL)

Ver fragmentación de índices

```
SELECT
  DB_NAME(database_id) AS BaseDeDatos,
  OBJECT_NAME(object_id) AS Tabla,
  index_id, avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats
(DB_ID(), NULL, NULL, NULL, 'SAMPLED');
```

Reorganizar índice

```
ALTER INDEX NombreIndice ON NombreTabla
REORGANIZE;
```

Reconstruir índice

```
ALTER INDEX NombreIndice ON NombreTabla
REBUILD;
```

Actualizar estadísticas

```
UPDATE STATISTICS NombreTabla;
-- o
EXEC sp_updatestats;
```

¿SABÍAS QUÉ?

Así como los navegantes portugueses planificaban cada ruta para llegar más lejos, en SQL Server los índices y estadísticas son las herramientas que guían al motor de base de datos por el mejor camino para encontrar la información.



Portugal

3. ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

Las transacciones y los bloqueos en SQL Server garantizan la integridad y consistencia de los datos cuando múltiples usuarios acceden y modifican la información de manera concurrente.

¿QUÉ ES UNA TRANSACCIÓN?

Es un conjunto de operaciones que se ejecutan como una sola unidad lógica de trabajo. Todas las operaciones se completan con éxito o ninguna se realiza.



Ejemplo: Transferencia bancaria

1. Descontar dinero de la cuenta origen.
2. Agregar dinero a la cuenta destino.

Si falla uno de los pasos, toda la operación se cancela.

COMANDOS PRINCIPALES



BEGIN TRANSACTION

Inicia una transacción.



COMMIT

Confirma todos los cambios realizados en la transacción.



ROLLBACK

Cancela todos los cambios realizados en la transacción.

PROPIEDADES ACID



Atomicidad

Todo o nada. La transacción se completa o se revierte por completo.



Consistencia

Los datos permanecen válidos antes y después de la transacción.



Aislamiento

Las transacciones concurrentes no interfieren entre sí.



Durabilidad

Después del COMMIT, los datos permanecen guardados incluso ante fallos del sistema.



¿QUÉ ES UN BLOQUEO?

Cuando un usuario accede o modifica un dato, SQL Server bloquea ese recurso para evitar conflictos y mantener la integridad de la información.

TIPOS DE BLOQUEO



SHARED (S)

Para lectura.

Permite que otros usuarios también lean el mismo recurso.



EXCLUSIVE (X)

Para escritura.

Ningún otro usuario puede leer o escribir el recurso mientras esté bloqueado.



UPDATE (U)

Para actualización.

Evita conflictos al convertir el bloqueo a exclusivo más adelante.

PROBLEMAS COMUNES

BLOCKING (BLOQUEO)

Una transacción debe esperar a que otra libere el recurso que necesita.



A espera a que B termine.

DEADLOCK (INTERBLOQUEO)

Dos o más transacciones se esperan mutuamente y ninguna puede continuar.



SQL Server detecta el problema y cancela una de las transacciones.



BUENAS PRÁCTICAS



Mantener transacciones lo más cortas posible.



Evitar dejar transacciones abiertas innecesariamente.



Crear índices adecuados para reducir bloqueos.



Actualizar solo los registros necesarios.



Monitorear bloqueos y deadlocks regularmente.

BENEFICIOS



Garantiza la integridad de los datos.



Permite el acceso concurrente seguro.



Evita pérdidas y corrupción de datos.



Mejora la confiabilidad del sistema.

EJEMPLO DE TRANSACCIÓN

```
BEGIN TRANSACTION
UPDATE Cuentas
SET Saldo = Saldo - 100,
WHERE IdCuenta = 1;

UPDATE Cuentas
SET Saldo = Saldo + 100
WHERE IdCuenta = 2;

COMMIT;
```

¡Éxito!



¿SABÍAS QUÉ?

SQL Server utiliza un mecanismo inteligente de bloqueo a diferentes niveles (fila, página, tabla) para maximizar la concurrencia y minimizar conflictos.



France

4. ANÁLISIS DE PLANES DE EJECUCIÓN

CÓMO SQL SERVER EJECUTA TUS CONSULTAS

El plan de ejecución muestra el camino que sigue SQL Server para obtener los datos. Analizarlo permite detectar problemas y mejorar el rendimiento de las consultas.

¿PARA QUÉ SIRVE?

- ⌚ Permite detectar consultas lentas.
- 🔍 Encuentra escaneos innecesarios.
- 💾 Identifica índices faltantes.
- 💰 Detecta operaciones costosas.
- ✅ Ayuda a optimizar el rendimiento.

TIPOS DE PLANES



PLAN ESTIMADO

No ejecuta la consulta. Solo muestra una estimación del costo y el comportamiento.



PLAN REAL

Ejecuta la consulta y muestra exactamente lo que ocurrió, con estadísticas reales.

OPERADORES COMUNES



TABLE SCAN

Lee toda la tabla. Generalmente es lento.



INDEX SEEK

Busca directamente usando índice. Muy eficiente.



INDEX SCAN

Lee gran parte del índice.



NESTED LOOP

Une tablas pequeñas. Eficiente en pocos registros.



HASH MATCH

Une tablas grandes. Usa más memoria.



SORT

Ordena resultados. Puede consumir mucha memoria.

¿QUÉ ANALIZAR EN EL PLAN?

- 📊 Operadores con mayor porcentaje de costo.
- ⚠️ Advertencias (Warnings).
- 💾 Índices faltantes sugeridos.
- ➕ Conversiones implícitas.
- 📈 Número de filas estimadas vs. reales.
- 💻 Uso de memoria y tiempo de CPU.



VISUALIZACIÓN DEL PLAN



SELECT



INDEX SEEK



NESTED LOOP



KEY LOOKUP



SORT

— Más eficiente

— Menos eficiente (mayor costo)

EJEMPLO DE PLAN DE EJECUCIÓN

```
SELECT c.Nombre, o.Total
FROM Clientes c
JOIN Ordenes o ON c.IdCliente = o.IdCliente
WHERE o.Fecha >= '2024-01-01';
```



Interpretación:

La mayor parte del costo está en el INDEX SEEK de Ordenes. Revisar índices y filtros puede mejorar el rendimiento.

BENEFICIOS

- 🚀 Optimiza consultas.
- ⌚ Reduce tiempos de ejecución.
- 🔧 Mejora el uso de recursos.
- 👥 Aumenta la capacidad de usuarios concurrentes.

BUENAS PRÁCTICAS

- 📈 Mantener estadísticas actualizadas.
- 💾 Crear índices adecuados.
- ❌ Evitar SELECT *.
- 🔄 Probar cambios y volver a evaluar el plan.
- 🔍 Usar filtros específicos.

La optimización es el arte de hacer más con menos, como en un buen plato italiano.

"Chi ben comincia è a metà dell'opera."

(Quien bien empieza, está a mitad del camino.)

¿SABÍAS QUÉ?

Un buen plan de ejecución puede hacer que una consulta que tarda minutos, ahora tarde milisegundos.



5. OPTIMIZACIÓN DE CONSULTAS T-SQL

CONSULTAS MÁS EFICIENTES, SISTEMAS MÁS RÁPIDOS

Optimizar una consulta significa obtener los mismos resultados utilizando menos recursos, en menos tiempo y con mayor eficiencia. Pequeños cambios pueden generar grandes mejoras.

¿POR QUÉ OPTIMIZAR?

- Menor tiempo de respuesta.
- Menor uso de CPU y memoria.
- Menor consumo de disco.
- Mayor cantidad de usuarios concurrentes.
- Mejor rendimiento general del sistema.

BUENAS PRÁCTICAS GENERALES

- Conoce tus datos y tu modelo.
- Usa índices adecuados.
- Escribe solo lo necesario.
- Evita operaciones innecesarias.
- Analiza y prueba siempre tus consultas.

最適化

ÍNDICES: TU MEJOR ALIADO

Crea índices en las columnas usadas en:

- WHERE
- JOIN
- ORDER BY
- GROUP BY

Los índices permiten que SQL Server encuentre los datos rápidamente sin revisar toda la tabla.

TÉCNICAS DE OPTIMIZACIÓN

1. EVITAR SELECT *

Trae solo las columnas necesarias.

❌ No recomendado

```
SELECT *
FROM Clientes;
```

✅ Recomendado

```
SELECT Nombre, Apellido, Email
FROM Clientes;
```

2. FILTRAR CORRECTAMENTE

Usa WHERE para reducir la cantidad de registros.

❌ No recomendado

```
SELECT *
FROM Ventas;
```

✅ Recomendado

```
SELECT *
FROM Ventas
WHERE Fecha >= '2024-01-01';
```

3. EVITAR FUNCIONES EN COLUMNAS INDEXADAS

Las funciones impiden el uso eficiente del índice.

❌ No recomendado

```
SELECT *
FROM Pedidos
WHERE YEAR(Fecha) = 2025;
```

✅ Recomendado

```
SELECT *
FROM Pedidos
WHERE Fecha >= '2025-01-01'
AND Fecha < '2026-01-01';
```

4. USAR JOIN CORRECTAMENTE

Usa JOIN explícitos y adecuados.

❌ No recomendado (sintaxis implícita)

```
SELECT *
FROM A, B
WHERE A.Id = B.Id;
```

✅ Recomendado (JOIN explícito)

```
SELECT *
FROM A
INNER JOIN B ON A.Id = B.Id;
```

5. USAR EXISTS CUANDO CORRESPONDA

Mejor para verificar existencia de datos.

❌ No recomendado

```
SELECT *
FROM Clientes c
WHERE c.Id IN (SELECT IdCliente
FROM Pedidos);
```

✅ Recomendado

```
SELECT *
FROM Clientes c
WHERE EXISTS (SELECT 1
FROM Pedidos p
WHERE p.IdCliente = c.Id);
```

6. EVITAR CURSORES

Los cursores procesan fila por fila (lentos).
Prefiere operaciones por conjuntos.

❌ No recomendado

```
-- Usar cursor para recorrer registros
```

✅ Recomendado

```
-- Usar sentencias SET BASED
(INsert, UPDATE, DELETE, SELECT)
```

OTROS CONSEJOS IMPORTANTES

- Evita conversiones implícitas de datos.
- No uses TOP sin ORDER BY.
- Usa tipos de datos adecuados.
- Actualiza estadísticas regularmente.
- Revisa y elimina código innecesario.
- Prueba diferentes alternativas y quédate con la mejor.

ANTES Y DESPUÉS

Antes (consulta lenta)

```
SELECT *
FROM Ordenes o, Clientes c
WHERE o.IdCliente = c.Id
AND YEAR(o.Fecha) = 2025
AND UPPER(c.Ciudad) = 'TOKYO';
```

Tiempo de ejecución: 5.28 seg
Lecturas lógicas: 125,000
CPU: Alto

Después (consulta optimizada)

```
SELECT o.IdOrden, o.Fecha, c.Nombre, c.Ciudad
FROM Ordenes o
INNER JOIN Clientes c ON o.IdCliente = c.Id
WHERE o.Fecha >= '2025-01-01'
AND o.Fecha < '2026-01-01'
AND c.Ciudad = 'Tokyo' COLLATE Latin1_General_CI_AS;
```

Tiempo de ejecución: 0.18 seg
Lecturas lógicas: 1,250
CPU: Bajo

BENEFICIOS

- Consultas más rápidas.
- Menor uso de recursos.
- Mejor experiencia para el usuario.
- Mayor estabilidad en la base de datos.
- Sistemas preparados para crecer.

¿SABÍAS QUÉ?

En japonés, la palabra "Kaizen" (改善) significa "Mejora continua". La optimización de consultas es un proceso constante de pequeños cambios que generan grandes resultados.

継続は力なり

(KEIZOKU WA CHIKARA NARI)

La constancia es la clave del éxito.

6. CONTROL DE RECURSOS CON RESOURCE GOVERNOR

ADMINISTRA, LIMITA Y PRIORIZA EL USO DE RECURSOS EN SQL SERVER

Resource Governor permite controlar cómo SQL Server distribuye CPU, memoria y conexiones entre diferentes usuarios o aplicaciones para mantener el rendimiento y la estabilidad del sistema.

¿QUÉ ES RESOURCE GOVERNOR?

Es una característica de SQL Server que administra los recursos del sistema para evitar que una carga de trabajo consuma todos los recursos disponibles y afecte el rendimiento de otras.



¿QUÉ RECURSOS CONTROLA?



CPU

Limita el uso del procesador.



MEMORIA

Controla la memoria utilizada.



CONEXIONES

Limita el número de conexiones.



CARGAS DE TRABAJO

Administra diferentes tipos de trabajo.

COMPONENTES PRINCIPALES



RESOURCE POOL

Grupo de recursos del sistema. Define cuánto CPU y memoria puede usar un grupo de trabajo.



WORKLOAD GROUP

Agrupar usuarios o aplicaciones con características similares. Cada grupo usa un Resource Pool.



FUNCIÓN CLASIFICADORA

Determina automáticamente a qué Workload Group pertenece cada conexión según reglas definidas.



¿CÓMO FUNCIONA?



Un usuario o aplicación se conecta a SQL Server.

La función clasificadora analiza la conexión y la asigna a un Workload Group.

El Workload Group utiliza los recursos del Resource Pool asignado.

SQL Server distribuye los recursos según las reglas definidas, garantizando equilibrio.

BENEFICIOS



Evita que una consulta o aplicación monopolice el servidor.



Prioriza aplicaciones críticas y procesos importantes.



Mejora la estabilidad y el rendimiento general del sistema.



Permite administrar mejor los recursos entre muchos usuarios y aplicaciones.



Reduce tiempos de espera y mejora la experiencia del usuario.



EJEMPLO DE ASIGNACIÓN DE RECURSOS

RESOURCE POOL	CPU MÁXIMO	MEMORIA MÁXIMA	USO TÍPICO
Pool_OLTP (Transacciones)	60%	60%	Aplicaciones críticas, operaciones diarias.
Pool_Reportes (Reportes)	30%	30%	Reportes y consultas analíticas.
Pool_Background (Procesos en segundo plano)	10%	10%	Tareas de mantenimiento, procesos programados.

CASOS DE USO



Empresas con muchos usuarios conectados al mismo tiempo.



Servidores con varias aplicaciones compartiendo la misma instancia de SQL Server.



Sistemas donde ciertos procesos requieren mayor prioridad.



EJEMPLO BÁSICO EN T-SQL

```
-- 1. Crear Resource Pool
CREATE RESOURCE POOL Pool_OLTP
WITH (MAX_CPU_PERCENT = 60, MAX_MEMORY_PERCENT = 60);

-- 2. Crear Workload Group
CREATE WORKLOAD GROUP WG_OLTP
USING Pool_OLTP;

-- 3. Crear función clasificadora (ejemplo simple)
CREATE FUNCTION dbo.fn_Classifier()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    RETURN 'WG_OLTP';
END;

-- 4. Crear Resource Governor
CREATE RESOURCE GOVERNOR MyResourceGovernor
WITH (CLASSIFIER_FUNCTION = dbo.fn_Classifier);
ALTER RESOURCE GOVERNOR MyResourceGovernor WITH (STATE = ON);
```



RECOMENDACIONES



Monitorea continuamente el uso de recursos.



Ajusta los porcentajes según las necesidades del sistema.



Prueba las reglas antes de aplicarlas en producción.



Combina Resource Governor con índices, estadísticas y buenas prácticas de consultas.



Documenta y revisa las configuraciones periódicamente.



¿SABÍAS QUÉ?

Resource Governor es como ser el "árbitro" de un partido de fútbol: distribuye el tiempo (CPU) y el espacio (memoria) de manera justa para que todos los equipos (usuarios y aplicaciones) puedan jugar sin que uno solo acapare todo.

